```
    var buffer =
ByteBuffer.allocate(ivLength).order(ByteOrder.BIG_ENDIAN);
    buffer.put((byte) id.length);    #C
    buffer.put(new byte[idLen - id.length]);    #C
    buffer.put(id);    #C
    buffer.put((byte) ((sequenceNumber >>> 32) & 0xFF));
#D
    buffer.putInt((int) sequenceNumber);    #D
    return xor(buffer.array(), commonIv);    #E
}
```

#A Check the sequence number is not too large
#B Check the Sender ID fits in the remaining space
#C Encode the Sender ID length, followed by the Sender ID left-padded to 6 less than the IV length
#D Encode the sequence number as a 5-byte big-endian integer
#E XOR the result with the Common IV to derive the final nonce

ENCRYPTING A MESSAGE

Once you've derived the per-message nonce, you can encrypt an OSCORE message, as shown in listing 13.12, which is based on the example in section C.4 of Appendix C of the OSCORE specification. OSCORE messages are encoded as COSE_Encrypt0 structures, in which there is no explicit recipient information. The Partial IV and the Sender ID are encoded into the message as *unprotected* headers, with the Sender ID using the standard COSE Key ID (KID) header. Although marked as unprotected, those values are actually authenticated because OSCORE requires them to be included in a COSE *external additional authenticated data*